

TC Hostel Connect: Project Briefing

Audience: Management **Review date:** 28 September 2026 **Purpose:** Explain the product, technology, system connections, current readiness, and decisions needed before launch.

Executive Summary

TC Hostel Connect is a web-based hostel operations platform intended to bring student, staff, warden, and administrator workflows into one system. Its feature areas include student and room records, leave approvals, attendance, visitor logs, maintenance, staff tasks, messages, notifications, and reports. The repository also contains a company-level control-center service and an Android wrapper.

The project has a substantial implementation and a working technical foundation. In this review, the frontend production build succeeded, the API type-check succeeded, and all 100 tests in the backend route-test suite passed. These results show that key code paths build and that tested API behavior works under the test setup; they do **not** prove the deployed system, all screens, real institution data, security controls, or operational processes are production-ready.

Recommendation: Treat the product as a promising MVP/staging candidate, not as cleared for a live multi-institution launch. First validate real-data workflows end to end, resolve deployment configuration gaps, and complete security, privacy, backup, and user-acceptance checks. A previous repository report labels the product “85% production-ready,” but that report is dated 22 March 2026 and contains unchecked staging actions; its percentage should not be used as a current readiness measure.

Project Need and Intended Users

Hostel teams commonly coordinate accommodation, attendance, leave, visitors, maintenance, and student communications across separate registers, spreadsheets, and conversations. The intended need is a shared operational record with role-appropriate access, less duplicate data entry, faster approval and follow-up, and clearer visibility for hostel leadership.

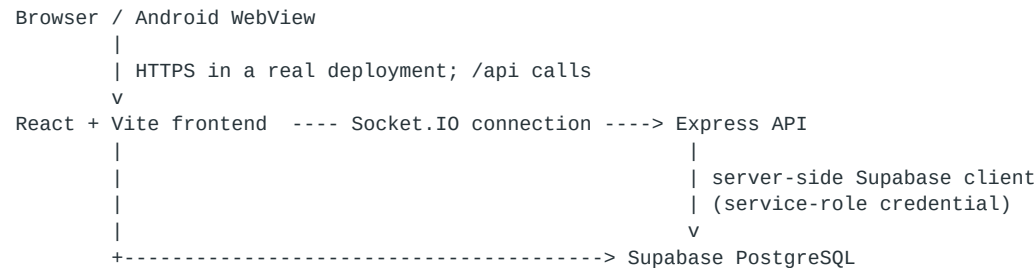
- 1 **Students:** view hostel information and submit or track requests such as leave and applications.
- 1 **Staff:** handle assigned operational work, attendance, and student support.
- 1 **Wardens:** oversee students, rooms, leave decisions, attendance, visitors, and maintenance.
- 1 **Institution administrators:** manage accounts and hostel operations and review reports.
- 1 **Platform/company administrators:** use the separate control-center service for cross-college governance functions.

These are the product’s intended outcomes. The repository does not establish measured reductions in workload, approval times, errors, or operating costs; those should be measured during a pilot.

Technology in Use

Area	Current implementation
Web frontend	React 18, TypeScript, Vite, React Router; Tailwind CSS and shared UI components. TanStack Query and Zustand are installed for data/cache and client state.
Backend/API	Node.js, TypeScript, Express REST endpoints; JWT-based authentication and role checks; Swagger/OpenAPI docs; Socket.IO for realtime notifications.
Database	Supabase-hosted PostgreSQL is the primary backend data store, accessed from the API using the Supabase JavaScript client. SQL schema/migrations define hostel and multi-college data. SQLite is present as a local-auth compatibility path, not the stated primary production database.
Mobile	Capacitor Android project wraps the web experience. This is a hybrid app foundation; store publication and device-specific validation are not demonstrated by this review.
Hosting/runtime	Frontend is a Vite-built static web application. The API is a separately run Node service (port 3001 in local configuration). Supabase hosts the managed database. Docker files support local container orchestration, but the checked-in compose configuration runs development commands.
Quality tooling	Vitest for unit/route tests, Playwright for browser journeys, and k6 scripts for load testing. The existence of these tools does not mean every suite has recently passed.

How the Pieces Connect



In local development, Vite is configured to proxy `/api` requests to `http://localhost:3001`. The Express service registers routes for authentication, students, rooms, leave, attendance, staff, maintenance, visitors, messages, reports, applications, and notifications. It validates JWTs and applies role checks in middleware. The API also serves health and Swagger documentation endpoints. Socket.IO is attached to the API's HTTP server for live events.

The API's database client requires `SUPABASE_URL` and `SUPABASE_SERVICE_ROLE_KEY`. That key must remain on the server and in a production secret store; it must never be exposed in browser build variables. The schema includes users, students, staff, rooms/beds, attendance, leave requests, visitors, maintenance, messages, applications, events, and task/activity tables. Migrations enable Row Level Security on tables, but application access through the service-role key bypasses normal RLS enforcement. Tenant boundaries therefore also depend on correct authorization and `college_id` filtering in every API query.

Important implementation distinction: The application is not yet consistently backed by live API data. Authentication attempts the API but supports mock/demo fallback. The student-management screen explicitly uses `mockStudents` for listing, filtering, edits, and deletion, and labels its backend list call as a TODO. Other screens also import the shared mock-data store. A successful API test suite therefore cannot be treated as proof that all user-visible workflows persist to the shared database.

Readiness Assessment

Verified in this review

- 1 `npm run build:web` completed successfully. Vite reported a main JavaScript chunk of about 1,069 KB before gzip (about 282 KB gzipped), above the configured 600 KB warning threshold. The build also reported that mock-data is both statically and dynamically imported and that the Supabase chunk is empty.
- 1 `npm run typecheck:api` completed successfully.
- 1 `npm --prefix services/api run test:api` passed: 8 test files, 100 tests.
- 1 API route coverage, JWT/role middleware, Supabase migrations, frontend routes, and deployment templates exist in the repository.

Not demonstrated by these checks

- 1 A working production/staging environment connected to an institution's Supabase project and migrated schema.
- 1 Full browser journeys using real accounts and persistent data; a current Playwright run was not performed for this review.
- 1 A current load-test result, backup/restore exercise, disaster-recovery target, or production monitoring/alert verification.
- 1 Complete tenant-isolation testing, independent security review, privacy/legal approval, accessibility sign-off, or mobile store/device release readiness.

Priority launch risks and actions

- 1 **Finish real-data workflows.** Replace mock-backed reads and mutations in each launch-critical screen with authenticated API calls. Make demo mode explicit and impossible to mistake for saved production data. Test that changes persist after reload and appear to another authorized user.
- 2 **Verify tenant isolation.** Review every endpoint for role authorization and `college_id` scoping, including joined records. Test that a user from one college cannot read or mutate another college's data. Do not rely on RLS alone while the API uses a service-role key.
- 3 **Fix and test production CORS.** The frontend sends an `x-college-id` header, while the API's configured allowed headers list includes only `Content-Type` and `Authorization`. Separate-origin browser deployments may fail preflight requests. Add only the required header(s), configure exact production origins, and test the deployed browser-to-API flow.
- 4 **Harden configuration and deployment.** Do not use the example JWT secret or placeholder Supabase values. Store secrets in the hosting provider's secret manager, enable HTTPS, use production logging, and confirm the actual hosting topology. The checked-in Compose file and portions of the README describe development setup, not a complete hardened production deployment.
- 5 **Protect sensitive records.** Student, guardian, emergency-contact, medical-note, attendance, and visitor information is sensitive. Define least-privilege access, retention/deletion, backup encryption, audit access, incident response, privacy notice/consent, and applicable legal requirements before onboarding real students.
- 6 **Complete release evidence.** Run end-to-end tests against staging with seeded test accounts, execute representative load tests, verify backup restoration and monitoring alerts, and obtain warden/staff/student acceptance before a controlled pilot.
- 7 **Reduce bundle and script ambiguity.** Address the oversized main frontend chunk and review unnecessary/empty chunk configuration. Align README commands with the actual root and package scripts: the root `dev` and `build` scripts currently invoke Vite directly, while the README describes workspace-wide orchestration.

Suggested Rollout

- 1 **Stage 1: Engineering staging.** Complete environment setup, migrations, CORS, tenant/security review, and critical real-data screens. Keep real student data out until controls are verified.
- 1 **Stage 2: Controlled pilot.** Pilot with one hostel/college and a small user group using agreed workflows, support ownership, data handling rules, and rollback procedures. Track task completion, failed requests, approval time, and support issues.
- 1 **Stage 3: Broader launch decision.** Expand only after pilot acceptance, no unresolved high-severity security/privacy issues, verified backups and monitoring, and measured capacity for expected usage.

Management Decisions Needed

- 1 Confirm the first launch institution, user groups, and workflows that define MVP scope.
- 1 Name the business owner and operational support/escalation contact for launch.
- 1 Confirm hosting provider, production domains, region/data residency, and budget for Supabase and application hosting.
- 1 Assign owners for security/privacy review, legal requirements, backup/restore, and user acceptance.
- 1 Approve a pilot only after the launch conditions above have evidence and an accountable sign-off.

Repository Evidence

- 1 Project README
- 1 Root package scripts, web package, and API package
- 1 API server and route registration, Supabase client, and auth/tenant middleware
- 1 Vite API proxy, frontend auth integration, and student screen
- 1 Database migrations
- 1 Prior market-readiness report and staging checklist; both are historical planning documents, not current launch sign-off.